

CS601T: Deep Learning

Programming Assignment II

Fully Connected Neural Networks for Classification and Regression

Sameer Chakrawarti CS24BT023 Affan Pathan CS24BT033
Sudhakar Dalawayi CS24BT008

7 September 2026

Abstract

This report presents the design, implementation, and empirical evaluation of Fully Connected Neural Networks (FCNNs) built from scratch for four datasets: two classification tasks (linearly separable and nonlinearly separable) and two regression tasks (univariate and bivariate). All models are trained using Stochastic Gradient Descent (SGD) with squared error loss. Different network architectures are evaluated using a held-out validation set, and the best-performing architecture is selected based on validation accuracy. On the linearly separable dataset, a minimal 2-node hidden layer achieves perfect test accuracy (100%). On the nonlinearly separable dataset, the best architecture, with two hidden layers of 32 and 16 nodes, achieves 98.00% validation accuracy and 99.67% test accuracy. The smaller configurations exhibit different optimization behaviour, converging to solutions that predict only a single class. For both regression datasets, the FCNN achieves substantially lower test %RMSE than the single linear neuron used in Assignment 1, demonstrating improved modelling capability. An exploratory data analysis is also performed before presenting the modelling results.

Contents

1	Introduction	3
2	Exploratory Data Analysis	3
2.1	Dataset Overview	3
2.2	Linearly Separable Classification Dataset	3
2.3	Nonlinearly Separable Classification Dataset	4
2.4	Univariate Regression Dataset	4
2.5	Bivariate Regression Dataset	4
3	Methodology	5
3.1	Notation	5
3.2	Data Preparation	5
3.3	FCNN Architecture	5
3.4	Mathematical Formulation	7
3.5	Training Configuration and Model Selection	8
4	Classification Results	8
4.1	Dataset 1: Classification of Linearly Separable Data	8
4.2	Dataset 2: Classification of Nonlinearly Separable Data	13
4.3	Comparison with Assignment 1: Classification	19
5	Regression Results	19
5.1	Architecture Selection and RMSE Comparison	19
5.2	Dataset 1: Univariate Regression	20
5.3	Dataset 2: Bivariate Regression	22
5.4	Comparison with Assignment 1 — Regression	25
5.5	Inference and Observations from Regression Results	26
6	Conclusion	26
A	Dataset 1 Regression: Remaining Hidden Node Surfaces	26
B	Appendix: NLS Dataset 2 Additional Node Surfaces (32,16)	27
C	Dataset 2 Regression: Remaining Hidden Node Surfaces	29

1 Introduction

This assignment investigates the use of Fully Connected Neural Networks (FCNNs) for classification on four Group 27 datasets with different characteristics. The study examines how the network architecture, particularly the number of hidden layers and hidden nodes, affects classification performance. The models are implemented and trained from scratch using Stochastic Gradient Descent (SGD) and squared error loss.

The primary objectives are:

- Implement an FCNN framework from scratch without using TensorFlow, PyTorch, or equivalent deep learning libraries.
- Train the networks using SGD and squared error loss.
- Evaluate different network architectures and select the best architecture based on validation-set performance.
- Report the required classification metrics, confusion matrices, and visualisations for the selected architectures.
- Analyse the learned decision regions and model outputs for the selected architectures.
- Compare the FCNN performance with the single-neuron results obtained in Assignment 1.

2 Exploratory Data Analysis

2.1 Dataset Overview

Table 1 summarises all four datasets. All splits follow the 60/20/20 (train/validation/test) ratio. For classification, the data is split independently within each class into training, validation, and test sets using a 60/20/20 split. No missing values, NaN, or Inf entries were found in any dataset.

Table 1: Dataset summary (Group 27).

Dataset	Task	N	Features	Train / Val / Test
LS Classification	Classification	1,500	2	900 / 300 / 300
NLS Classification	Classification	2,000	2	900 / 300 / 300
Univariate Regression	Regression	1,001	1	600 / 200 / 201
Bivariate Regression	Regression	10,201	2	6,120 / 2,040 / 2,041

2.2 Linearly Separable Classification Dataset

The LS dataset contains 1,500 two-dimensional samples distributed equally across three classes (500 each). Table 2 provides per-class descriptive statistics.

Table 2: LS dataset: per-class feature statistics.

Class	Feature	Min	Max	Mean	Std	Median
0	x_1	7.11	25.64	15.84	3.14	15.86
	x_2	-4.58	4.43	0.10	1.59	0.16
1	x_1	1.35	12.08	6.00	1.74	5.93
	x_2	-18.73	-6.58	-12.01	2.15	-11.99
2	x_1	-5.22	3.27	-1.07	1.27	-1.06
	x_2	-6.79	9.26	1.70	3.01	1.70

Inter-centroid Euclidean distances range from 15.4 to 17.0, while within-class spread ranges from 2.4 to 3.0. The large separation-to-spread ratio confirms the classes are geometrically well separated; near-perfect accuracy is expected from any reasonable classifier.

2.3 Nonlinearly Separable Classification Dataset

The NLS dataset contains 2,000 two-dimensional samples across three classes. A radial distance analysis reveals a nested ring structure: Class 0 forms a tight inner ring near $r \approx 1.33$, Class 1 is a broader crescent at $r \in [0.04, 2.59]$, and Class 2 forms a large outer ring at $r \in [3.12, 5.20]$. For all three classes, circular boundaries are required to separate them.

2.4 Univariate Regression Dataset

The dataset contains 1,001 samples with a single input $x \in [0, 1]$ and a continuous target y . Table 3 provides descriptive statistics.

Table 3: Univariate regression: descriptive statistics.

	Min	Max	Mean	Std	Median
x	0.000	1.000	0.500	0.289	0.500
y	-3.754	-1.177	-2.736	0.826	-2.996

The Pearson correlation $r(x, y) = -0.745$ suggests a global downward trend, but binned analysis (Table 4) reveals a non-monotonic inverted-U: y rises from -2.62 near $x = 0$ to a peak of -1.32 near $x = 0.25$, then falls to -3.63 near $x \approx 0.75$. A linear model would miss this peak entirely.

Table 4: Univariate regression: binned mean y by x range.

x Range	n	Mean y	Std y
[0.00, 0.10]	100	-2.624	0.246
[0.10, 0.20]	100	-1.776	0.239
[0.20, 0.30]	101	-1.323	0.068
[0.30, 0.40]	99	-1.778	0.233
[0.40, 0.50]	100	-2.620	0.232
[0.50, 0.60]	101	-3.256	0.136
[0.60, 0.70]	100	-3.559	0.072
[0.70, 0.80]	99	-3.633	0.055
[0.80, 0.90]	100	-3.547	0.071
[0.90, 1.00]	101	-3.247	0.137

2.5 Bivariate Regression Dataset

The dataset contains 10,201 samples on a regular 101×101 grid with $x_1, x_2 \in [0, 1]$ and a continuous target y . Pearson correlations are $r(x_1, y) \approx 0.0002$ and $r(x_2, y) = -0.201$. Binned analysis reveals a symmetric U-shape along x_1 (mean y drops from ≈ 7.39 at edges to 3.64 at centre) and a weak negative linear trend along x_2 . A linear model captures the x_2 trend but assigns near-zero weight to x_1 , missing its dominant nonlinear contribution.

3 Methodology

3.1 Notation

Table 5 summarises the symbols used throughout this section.

Table 5: Summary of notation.

Symbol	Definition
L	Total number of layers (hidden + output)
l	Layer index, $l \in \{1, \dots, L\}$
N	Number of training examples in one epoch
K	Number of output nodes
n	Training example index, $n \in \{1, \dots, N\}$
k	Output node index, $k \in \{1, \dots, K\}$
$\mathbf{x}$	Input feature vector
$x_{\min}^{\text{train}}, x_{\max}^{\text{train}}$	Per-feature minimum and maximum over the training set
$\tilde{x}$	Min-max normalised input feature
$w_{ji}^{(l)}$	Weight from unit i in layer $l-1$ to unit j in layer l
$b_j^{(l)}$	Bias of unit j in layer l
$z_j^{(l)}$	Weighted input (pre-activation) of unit j in layer l
$a_j^{(l)}$	Activation output of unit j in layer l ; $a^{(0)} = \mathbf{x}$
$f^{(l)}(\cdot)$	Activation function of layer l
$\sigma(z)$	Logistic sigmoid: $1/(1 + e^{-z})$
$\hat{y}_{nk}$	Predicted output for example n , output node k
y_{nk}	Target value for example n , output node k
E_n	Squared error for training example n
$\bar{E}$	Average squared error over one training epoch
η	Learning rate
$\delta_j^{(l)}$	Error signal (delta) for unit j in layer l
$\Delta w_{ji}^{(l)}$	Weight update for $w_{ji}^{(l)}$

3.2 Data Preparation

All datasets are split 60/20/20. For classification the split is applied within each class. Target labels are one-hot encoded. Input features are normalised to $[0, 1]$ using min-max normalisation fitted exclusively on training data:

$$\tilde{x} = \frac{x - x_{\min}^{\text{train}}}{x_{\max}^{\text{train}} - x_{\min}^{\text{train}}} \quad (1)$$

3.3 FCNN Architecture

The FCNN is implemented from scratch using NumPy (framework: **synapse**). The **Layer** class absorbs the bias into the weight matrix. Networks are assembled by stacking layers of configurable width and activation.

Classification Models

Dataset 1 (LS): 2 inputs, one hidden layer, 3 outputs. Hidden sizes swept: $\{2, 4, 6, 8, 10\}$. Sigmoid activation on hidden and output layers. Figure 1 show representative architecture.

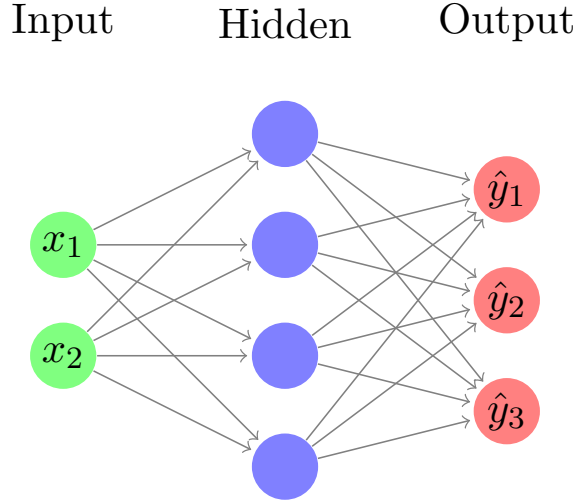


Figure 1: Dataset 1: single hidden layer (4 nodes shown).

Dataset 2 (NLS): 2 inputs, two hidden layers, 3 outputs. Configurations swept: (4,4),(8,4),(8,8),(16,8), (32,16). Sigmoid activation throughout. Figure 2 show representative architecture.

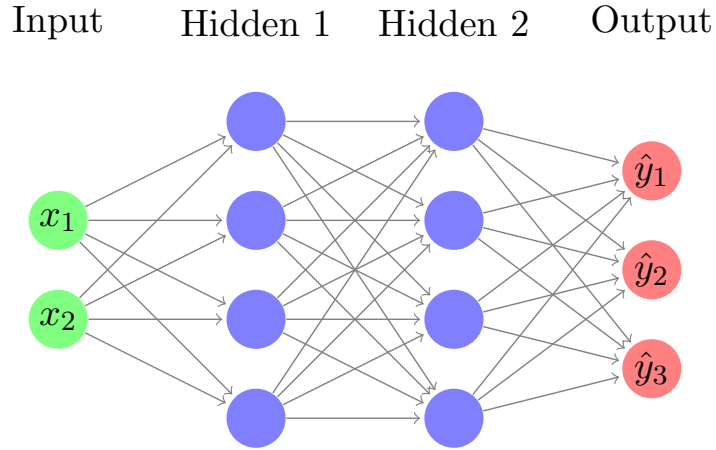


Figure 2: Dataset 2: two hidden layers (4→4 shown; best arch is 32→16).

Regression Models

Dataset 1 (Univariate): 1 input, one hidden layer, 1 output. Hidden sizes: {2, 4, 6, 8}. Sigmoid hidden, linear output.

Dataset 2 (Bivariate): 2 inputs, one or two hidden layers, 1 output. Configurations: (4),(8),(4,4),(8,4). Sigmoid hidden, linear output.

3.4 Mathematical Formulation

Forward Pass

For the j -th neuron in layer l , the weighted input and activation are given by:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)}, \quad a_j^{(l)} = f^{(l)}(z_j^{(l)}) \quad (2)$$

where $w_{ji}^{(l)}$ and $b_j^{(l)}$ denote the weights and bias, respectively.

Activation Function — Logistic Sigmoid

The hidden layers use the logistic sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (3)$$

For regression, the output layer uses a linear activation:

$$f(z) = z, \quad f'(z) = 1. \quad (4)$$

Loss Function

The squared error for training example n is:

$$E_n = \frac{1}{2} \sum_{k=1}^K (y_{nk} - \hat{y}_{nk})^2 \quad (5)$$

The average error over one training epoch, used to monitor convergence and reported in the error-vs-epoch plots, is:

$$\bar{E} = \frac{1}{N} \sum_{n=1}^N E_n, \quad (6)$$

where N is the number of training examples processed in that epoch.

SGD and Backpropagation

Weights are updated after each training example using stochastic gradient descent:

$$\Delta w_{ji}^{(l)} = \eta \delta_j^{(l)} a_i^{(l-1)} \quad (7)$$

For the output layer, the local gradient is:

$$\delta_k^{(o)} = (y_{nk} - \hat{y}_{nk}) f'^{(o)}(z_k^{(o)}) \quad (8)$$

For a hidden layer l , the error is backpropagated from layer $l + 1$:

$$\delta_j^{(l)} = \left[\sum_k \delta_k^{(l+1)} w_{kj}^{(l+1)} \right] f'^{(l)}(z_j^{(l)}) \quad (9)$$

Weights are initialized independently and randomly by sampling from $\mathcal{N}(0, 0.1^2)$.

Regression Evaluation Metrics

The regression models are evaluated using Root Mean Squared Error (RMSE) and Percentage RMSE (%RMSE).

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (10)$$

where N is the number of samples in the evaluated dataset split.

The percentage RMSE normalizes RMSE by the range of the actual target values:

$$\%RMSE = \frac{RMSE}{y_{\max} - y_{\min}} \times 100 \quad (11)$$

where $y_{\max}$ and $y_{\min}$ are the maximum and minimum actual target values in the corresponding dataset split.

3.5 Training Configuration and Model Selection

Table 6: Training hyperparameters.

Hyperparameter	Classification	Regression
Learning rate η	0.1	0.05
Max epochs	500	150
Early stopping patience	30	20
Early stopping Δ	10^{-6}	10^{-6}
Weight init std	0.1	0.1
Random seed	42	42

For Dataset 2, the original data contains 500 samples for each of Classes 1 and 2 and 1,000 samples for Class 3. All 500 available samples from Classes 1 and 2 were selected, while the first 500 samples from the 1,000 available Class 3 samples were selected, resulting in 500 samples per class and 1,500 samples in total. This equal-class sampling ensures a balanced training set, preventing the model from developing a bias towards the over-represented class during training.

Validation error is evaluated after each epoch. If the validation error fails to improve by more than $\Delta = 10^{-6}$ for 30 consecutive epochs in classification or 20 consecutive epochs in regression, training is terminated and the weights corresponding to the best validation error are retained.

For classification, the best architecture is selected based on maximum validation accuracy, while for regression it is selected based on minimum validation RMSE. The test set is used only for evaluating the selected architecture.

4 Classification Results

4.1 Dataset 1: Classification of Linearly Separable Data

Validation Results

All five architectures achieve perfect validation accuracy. The smallest, 2-node architecture, is selected of all models having the best validation accuracy.

Table 7: Dataset 1: validation metrics across all architectures.

Metric	2	4	6	8	10
Accuracy	1.0000	1.0000	1.0000	1.0000	1.0000
Precision – C0	1.0000	1.0000	1.0000	1.0000	1.0000
Precision – C1	1.0000	1.0000	1.0000	1.0000	1.0000
Precision – C2	1.0000	1.0000	1.0000	1.0000	1.0000
Mean Precision	1.0000	1.0000	1.0000	1.0000	1.0000
Recall – C0	1.0000	1.0000	1.0000	1.0000	1.0000
Recall – C1	1.0000	1.0000	1.0000	1.0000	1.0000
Recall – C2	1.0000	1.0000	1.0000	1.0000	1.0000
Mean Recall	1.0000	1.0000	1.0000	1.0000	1.0000
F-measure – C0	1.0000	1.0000	1.0000	1.0000	1.0000
F-measure – C1	1.0000	1.0000	1.0000	1.0000	1.0000
F-measure – C2	1.0000	1.0000	1.0000	1.0000	1.0000
Mean F-measure	1.0000	1.0000	1.0000	1.0000	1.0000

Table 8: Dataset 1: validation confusion matrices (all architectures identical).

True Class	Pred. 0	Pred. 1	Pred. 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

Test Results for Best Architecture

Table 9: Dataset 1: test metrics — best architecture (2 hidden nodes).

Metric	Value
Accuracy	1.0000
Precision – C0	1.0000
Precision – C1	1.0000
Precision – C2	1.0000
Mean Precision	1.0000
Recall – C0	1.0000
Recall – C1	1.0000
Recall – C2	1.0000
Mean Recall	1.0000
F-measure – C0	1.0000
F-measure – C1	1.0000
F-measure – C2	1.0000
Mean F-measure	1.0000

Table 10: Dataset 1: test confusion matrix — best architecture (2 hidden nodes).

True Class	Pred. 0	Pred. 1	Pred. 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

Error Convergence

Figure 3 illustrates the training convergence behavior of the best-performing architecture (2 hidden nodes) for Dataset 1. The average error begins at approximately 0.34 and experiences a rapid, steep decline between epochs 5 and 20. Following this sharp descent, the error curve stabilizes and approaches zero, demonstrating that the network efficiently learns the required decision boundaries and achieves smooth, stable convergence well before reaching 100 epochs.

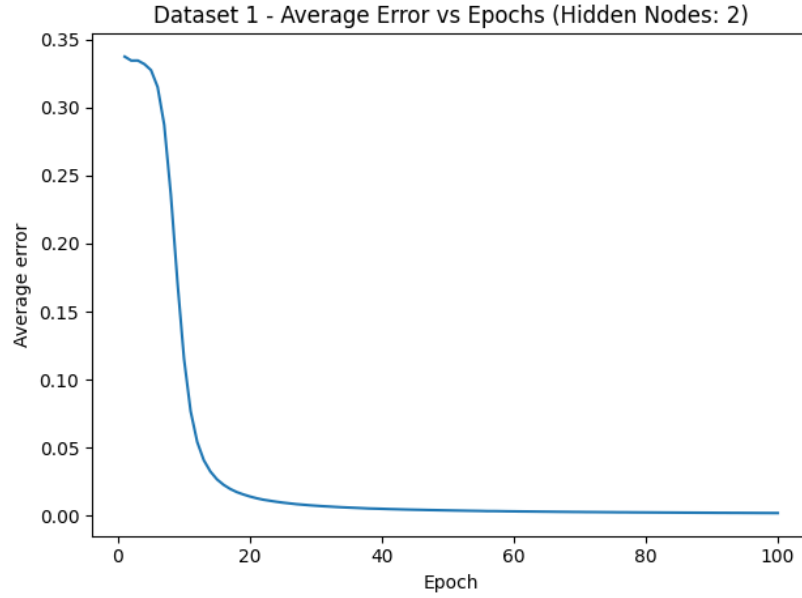


Figure 3: Dataset 1: average training error vs. epochs, best architecture (2 hidden nodes).

Decision Region

Figure 4 presents the decision regions established by the 2-hidden-node model superimposed on the training data. The network effectively partitions the feature space into three distinct zones with clear, near-linear boundaries, successfully isolating all three classes without any misclassifications.

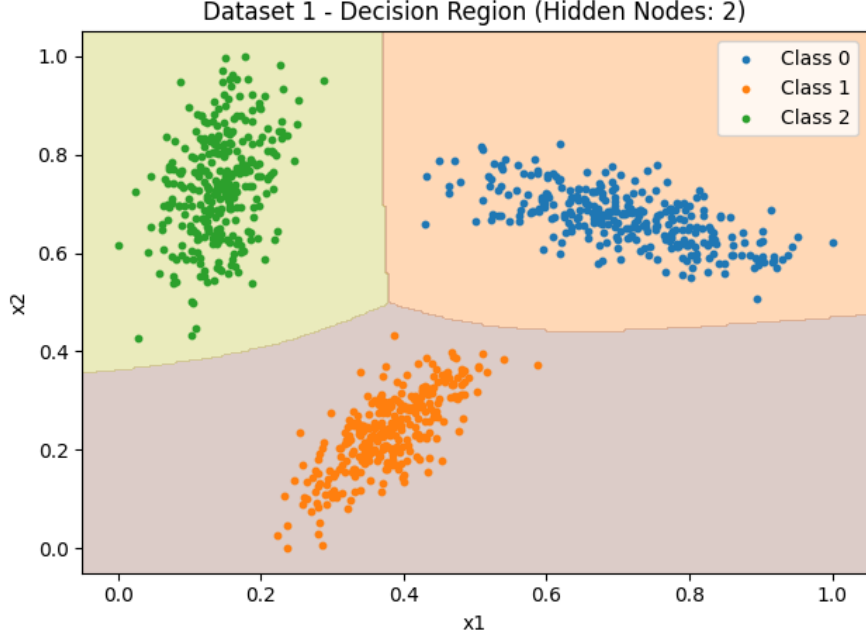


Figure 4: Dataset 1: decision region superimposed with training data (2 hidden nodes).

Node Output Surfaces

Figure 5 illustrates the activation surfaces for the best-performing architecture on the linearly separable dataset (Dataset 1), which consists of a single hidden layer with 2 nodes. In each 3D plot, the horizontal axes represent the two input features (x_1 and x_2), while the vertical z-axis displays the activation output of that specific node. The figure is organized into three columns corresponding to the training, validation, and test datasets, respectively.

These plots visualize the activation landscapes of the network’s neurons across the input feature space. The hidden nodes construct intermediate activation surfaces that transform the input features, while the output nodes act as class-specific indicators, forming regions with activation values close to 1.0 for their respective target classes and close to 0.0 elsewhere. The similarity of these surfaces across the training, validation, and test splits indicates that the learned representations are consistent across the different data splits.

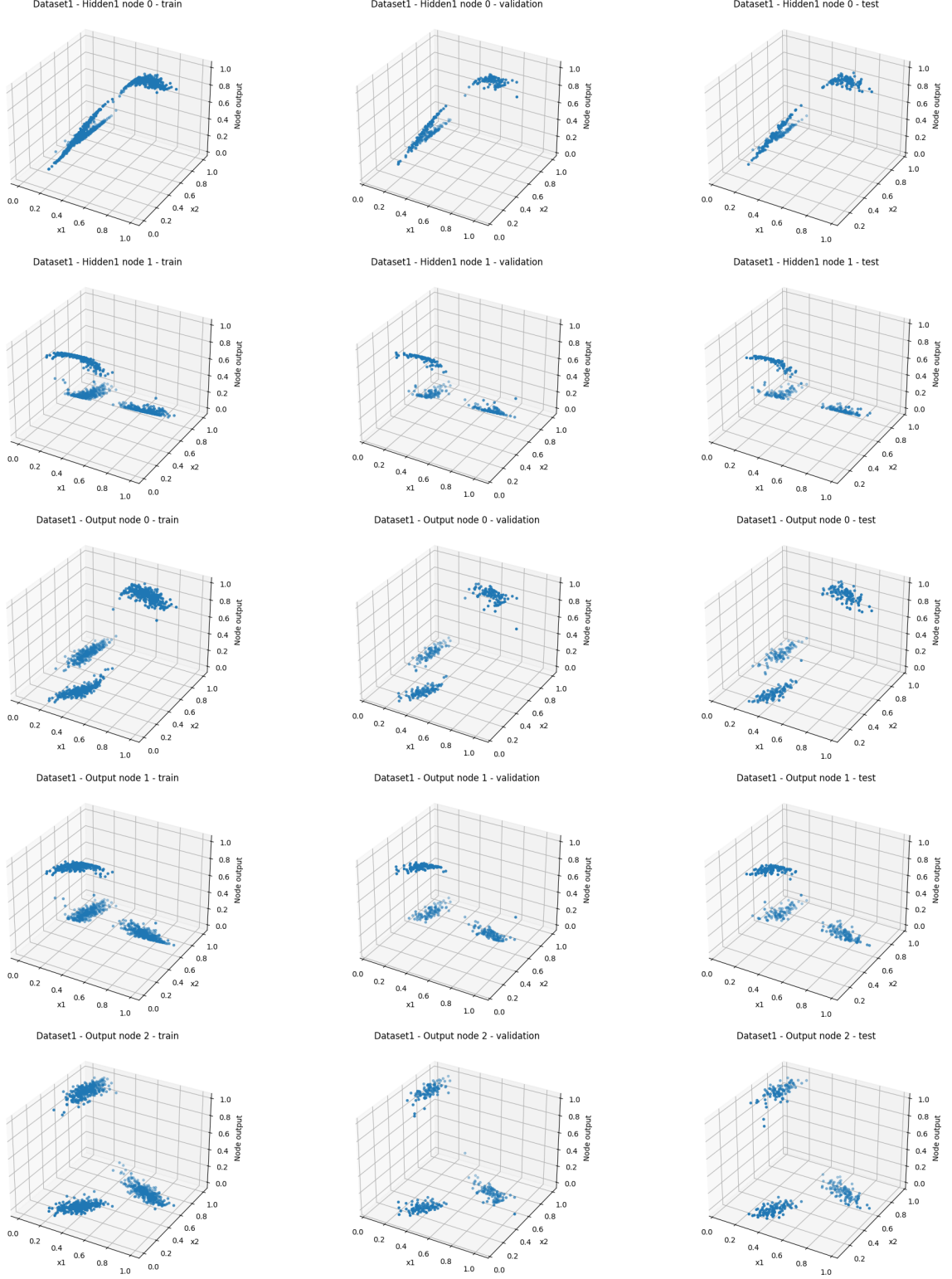


Figure 5: Dataset 1 (2 hidden nodes): node output surfaces for train (left), validation (centre), test (right). Rows: H1-node0, H1-node1, Out-node0, Out-node1, Out-node2.

Dataset 1 Analysis and Inferences

- All five architectures achieve 100% accuracy on both the validation and test sets, consistent with the strong inter-class separation indicated by the feature statistics in Table 2.

- Two hidden nodes suffice for Dataset 1: the network learns decision boundaries that cleanly separate the three well-spread Gaussian clusters. Increasing the number of hidden nodes does not improve the classification accuracy.
- The error curve (Figure 3) drops steeply and reaches a near-zero value within the first few epochs, indicating rapid convergence during training.
- The output node surfaces approach binary $\{0, 1\}$ values across the input space, consistent with the one-hot encoded targets and the clean separation of the three classes.

4.2 Dataset 2: Classification of Nonlinearly Separable Data

Validation Results

Table 11 presents the validation performance of the five evaluated architectures. The smaller configurations, (4, 4), (8, 4), (8, 8), and (16, 8), all achieve only 33.33% accuracy. Their per-class precision, recall, and F-measure values also show that these models fail to classify all three classes effectively, with each configuration predicting only one class consistently.

In contrast, the (32, 16) architecture achieves 98.00% validation accuracy, with mean precision, mean recall, and mean F-measure of 0.9800, 0.9800, and 0.9800, respectively. The substantially higher validation performance makes (32, 16) the best architecture among the evaluated configurations, and it is therefore selected for evaluation on the test set.

Table 11: Dataset 2: validation metrics across all architectures.

Metric	(4,4)	(8,4)	(8,8)	(16,8)	(32,16)
Accuracy	0.3333	0.3333	0.3333	0.3333	0.9800
Precision – C0	0.0000	0.0000	0.3333	0.0000	0.9700
Precision – C1	0.0000	0.3333	0.0000	0.0000	0.9700
Precision – C2	0.3333	0.0000	0.0000	0.3333	1.0000
Mean Precision	0.1111	0.1111	0.1111	0.1111	0.9800
Recall – C0	0.0000	0.0000	1.0000	0.0000	0.9700
Recall – C1	0.0000	1.0000	0.0000	0.0000	0.9700
Recall – C2	1.0000	0.0000	0.0000	1.0000	1.0000
Mean Recall	0.3333	0.3333	0.3333	0.3333	0.9800
F-measure – C0	0.0000	0.0000	0.5000	0.0000	0.9700
F-measure – C1	0.0000	0.5000	0.0000	0.0000	0.9700
F-measure – C2	0.5000	0.0000	0.0000	0.5000	1.0000
Mean F-measure	0.1667	0.1667	0.1667	0.1667	0.9800

Table 12 presents the validation confusion matrices for all five architectures, showing the distribution of predictions across the three classes for each true class. Each row corresponds to a true class, while each column represents the class predicted by the model. The values along the main diagonal indicate the number of correctly classified samples, whereas the off-diagonal values show the number of samples misclassified as another class. This allows the classification performance and the most common sources of confusion to be assessed.

Table 12: Dataset 2: validation confusion matrices.

Architecture	True Class	Pred. 0	Pred. 1	Pred. 2
(4,4)	Class 0	0	0	100
	Class 1	0	0	100
	Class 2	0	0	100
(8,4)	Class 0	0	100	0
	Class 1	0	100	0
	Class 2	0	100	0
(8,8)	Class 0	100	0	0
	Class 1	100	0	0
	Class 2	100	0	0
(16,8)	Class 0	0	0	100
	Class 1	0	0	100
	Class 2	0	0	100
(32,16)	Class 0	97	3	0
	Class 1	3	97	0
	Class 2	0	0	100

Test Results for the Best Architecture

Table 13 reports the test performance of the selected (32, 16) architecture, including overall accuracy and per-class as well as mean precision, recall, and F-measure.

Table 13: Dataset 2: test metrics — best architecture (32,16).

Metric	Best (32,16)
Accuracy	0.9967
Precision – C0	1.0000
Precision – C1	0.9901
Precision – C2	1.0000
Mean Precision	0.9967
Recall – C0	0.9900
Recall – C1	1.0000
Recall – C2	1.0000
Mean Recall	0.9967
F-measure – C0	0.9950
F-measure – C1	0.9950
F-measure – C2	1.0000
Mean F-measure	0.9967

Table 14 presents the test confusion matrix for the selected (32, 16) architecture, showing the correctly and incorrectly classified samples for each of the three classes.

Table 14: Dataset 2: test confusion matrix — best architecture (32,16).

True Class	Pred. 0	Pred. 1	Pred. 2
Class 0	99	1	0
Class 1	0	100	0
Class 2	0	0	100

Error Convergence

Figure 6 displays the average training error ($\bar{E}$, Eq. 6) over epochs for the optimal (32, 16) architecture on Dataset 2. The x-axis represents the training epochs, while the y-axis shows the average squared error computed on the training data between the network’s predicted outputs and the true one-hot encoded labels. The error decreases in a step-like manner, with further reductions after plateaus around epochs 100 and 420.

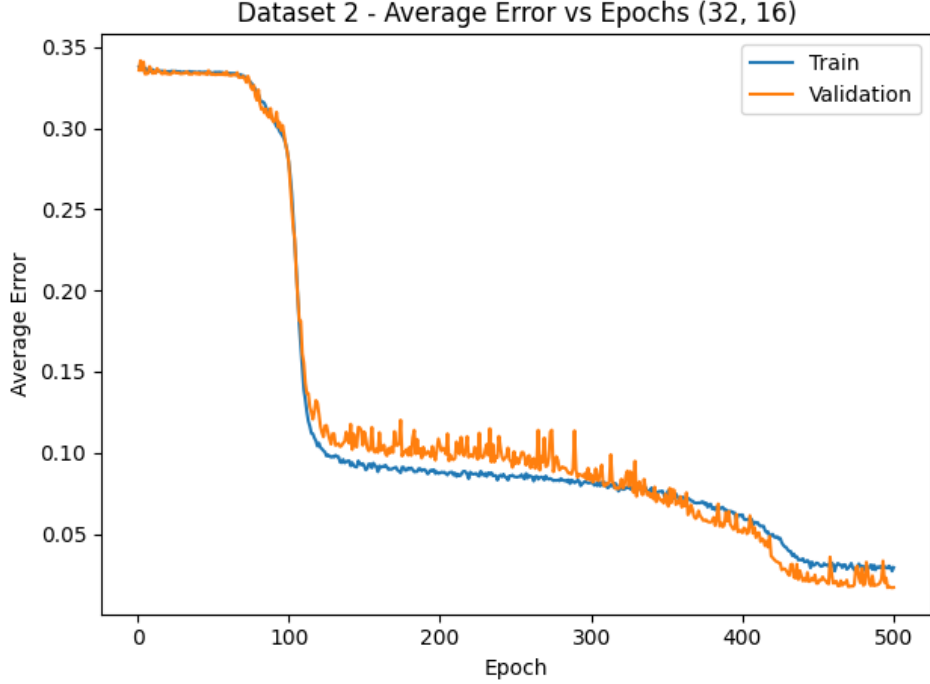


Figure 6: Dataset 2: average training error vs. epochs, best architecture (32,16).

Decision Region

Figure 7 visualizes the learned decision regions overlaid on the two-dimensional feature space (x_1 and x_2 axes). The background classification regions were generated by evaluating a dense 300×300 grid of points through the trained network and assigning each point to the class with the highest output activation ($\arg \max$). The training samples are then overlaid on these regions to show how the learned decision boundaries separate the three classes. The resulting non-linear decision boundaries demonstrate that the (32, 16) architecture, out of all, best captures the non-linear structure of Dataset 2.

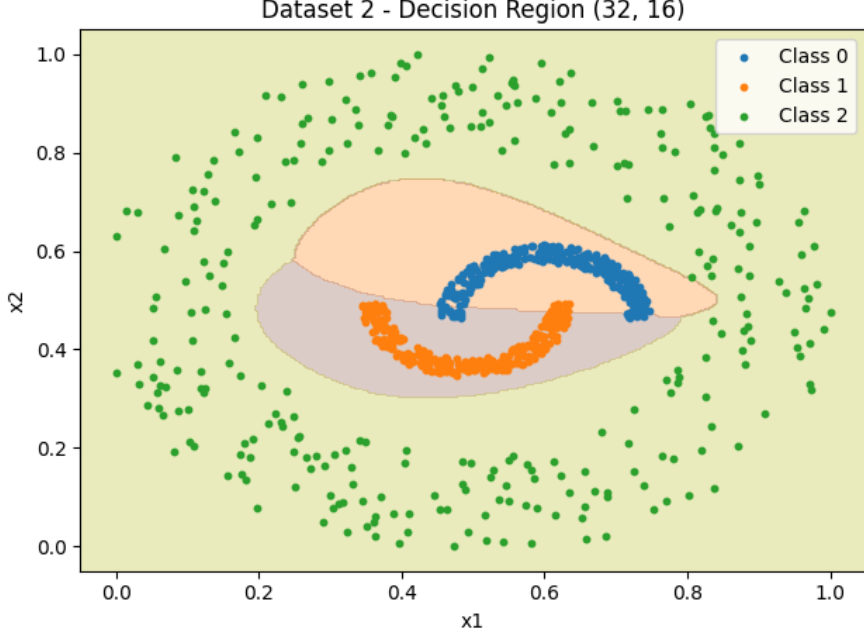


Figure 7: Dataset 2: decision region superimposed with training data, best architecture (32,16).

Node Output Surfaces

Figures 8, 9, and 10 illustrate a representative selection of the node activation surfaces for the best-performing architecture on Dataset 2, which consists of 32 nodes in the first hidden layer and 16 nodes in the second hidden layer.

In these three-dimensional plots, the horizontal x_1 and x_2 axes represent the two input features, while the vertical axis indicates the activation output of that specific node across the feature space. The visualizations are divided into separate figures corresponding to the training, validation, and test data splits.

Note: The full (32,16) network contains a total of 51 nodes (32 in the first hidden layer, 16 in the second, and 3 in the output layer), resulting in 153 individual plots across the three splits. For readability, only a representative subset is displayed in this section. The complete set of node surface plots is provided in Appendix II.

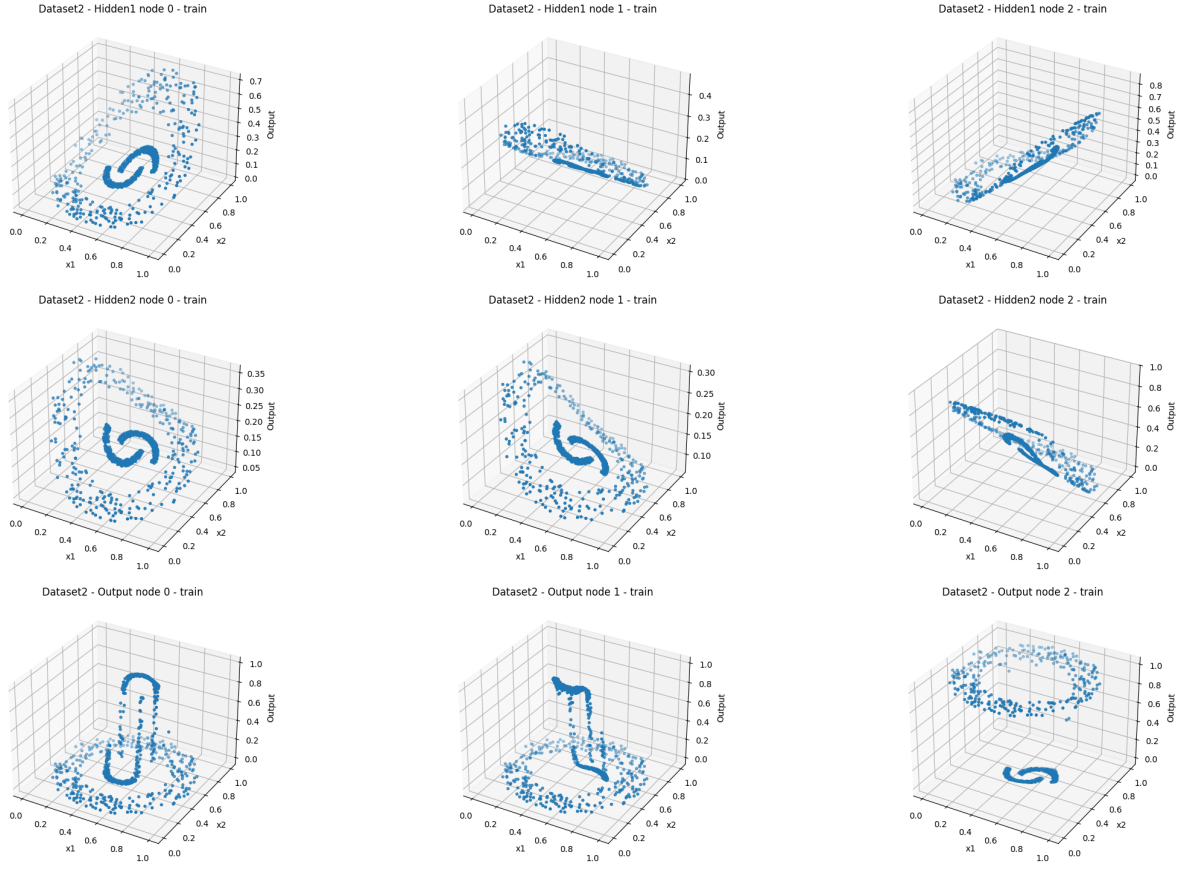


Figure 8: Dataset 2 (32,16): representative node surfaces on **training** data.

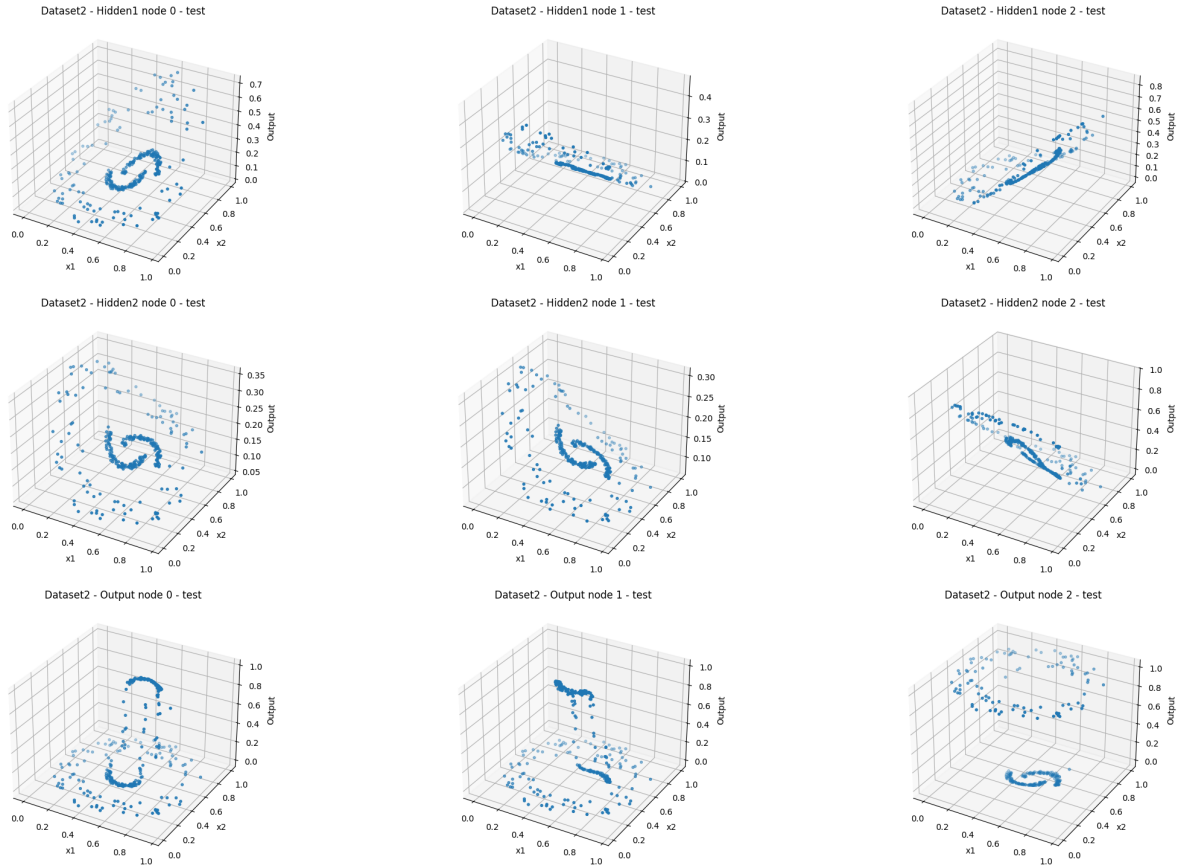


Figure 10: Dataset 2 (32,16): representative node surfaces on **test** data.

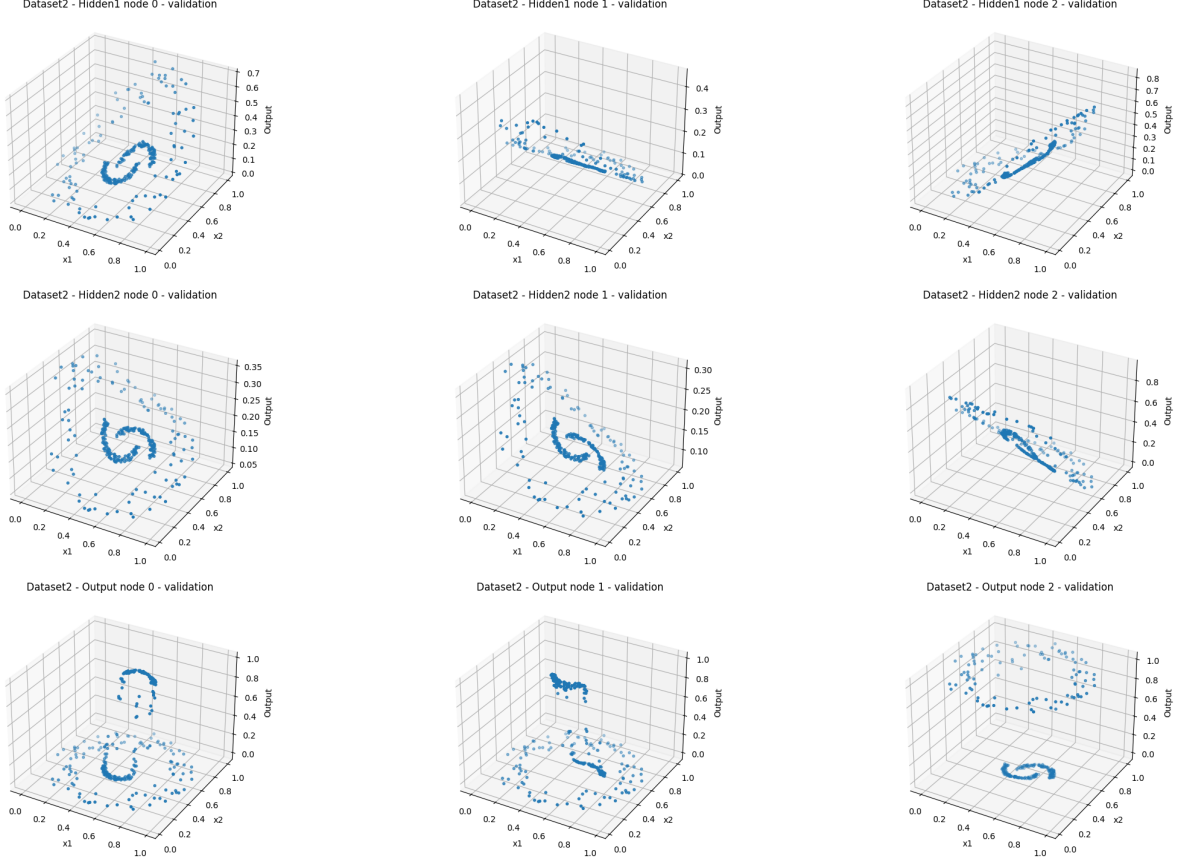


Figure 9: Dataset 2 (32,16): representative node surfaces on **validation** data.

Dataset 2 Analysis and Inferences

- As shown in Table 11, the architectures (4, 4), (8, 4), (8, 8), and (16, 8) achieve only 33.33% validation accuracy, with the corresponding confusion matrices in Table 12 showing that each model predicts a single class for all validation samples. In contrast, the (32, 16) architecture achieves 97.33% validation accuracy and successfully classifies all three classes. This large performance difference suggests that the additional hidden nodes provide sufficient representational capacity to learn the non-linear structure of Dataset 2.
- Figure 7 shows that the selected (32, 16) architecture learns non-linear decision regions that effectively separate the three classes in the two-dimensional feature space. The test results in Table 13 further support this, with 99.67% test accuracy and mean precision, recall, and F-measure all equal to 0.9967.
- The confusion matrix in Table 14 shows that only 1 test error occurs: 1 Class 0 sample is predicted as Class 1, while all other samples are correctly classified. This reflects the near-perfect separation learned by the (32, 16) architecture.
- Figure 6 shows that the average training error decreases over the 500 epochs, although the decrease occurs in several stages with plateaus before further reductions.

4.3 Comparison with Assignment 1: Classification

Table 15: Classification performance: Assignment 1 (single neuron) vs. Assignment 2 (FCNN).

Dataset	Model	Approach	Test Accuracy
Dataset 1 (LS)	Single Neuron (A1)	One-Against-One (3 binary classifiers)	1.0000
	FCNN (A2)	Unified 3-output network	1.0000
Dataset 2 (NLS)	Single Neuron (A1)	One-Against-One (linear boundaries)	0.4717
	FCNN (A2, 32,16)	Two hidden layers with sigmoid activations	0.9967

On Dataset 1, both models achieve perfect test accuracy. The FCNN uses a single unified 3-output network, whereas Assignment 1 uses three binary classifiers with a voting scheme.

On Dataset 2, the FCNN with architecture (32,16) achieves 99.67% test accuracy, compared with 47.17% for the best single-neuron approach (LogSigmoid, one-against-one). The decision-region visualization in Figure 7 shows that the FCNN learns non-linear decision regions suitable for separating the ring-structured classes, which is reflected in its substantially higher test performance.

5 Regression Results

5.1 Architecture Selection and RMSE Comparison

Table 16 summarizes the predictive performance of the Fully Connected Neural Network (FCNN) architectures evaluated on two regression tasks: Dataset 1 (univariate) and Dataset 2 (bivariate). For each architecture, the table reports the Root Mean Squared Error (RMSE) and percentage RMSE (%RMSE) on both the training and validation splits. You can refer to the The best-performing architecture for each dataset, selected based on the lowest validation RMSE, is highlighted in bold.

Table 16: RMSE and %RMSE on training and validation data for all architectures.

Dataset	Architecture	Train RMSE	Train %RMSE	Val RMSE	Val %RMSE
Dataset 1 (Uni.)	2 nodes	0.1127	4.382%	0.1046	4.060%
	4 nodes	0.0924	3.594%	0.0843	3.273%
	6 nodes	0.0904	3.514%	0.0831	3.226%
	8 nodes	0.0868	3.376%	0.0811	3.146%
Dataset 2 (Bi.)	4 nodes	0.0611	1.157%	0.0615	1.198%
	8 nodes	0.0608	1.151%	0.0610	1.189%
	8→4	0.0620	1.174%	0.0626	1.221%
	4→4	0.0611	1.156%	0.0610	1.189%

Table 17 presents the final evaluation metrics on the test data for the optimal architectures selected based on validation performance. For the univariate task (Dataset 1), the single-hidden-layer model with 8 nodes achieves a Test RMSE of 0.0851, corresponding to 3.424% RMSE relative to the target range. For the bivariate task (Dataset 2), the optimal two-hidden-layer model (4 → 4) achieves a Test RMSE of 0.0603, corresponding to 1.189% RMSE. The close agreement between the test RMSE values and the corresponding validation results indicates that both selected architectures generalize well to unseen test data, with no large difference between validation and test performance.

Table 17: RMSE and %RMSE on test data for the best architecture of each dataset.

Dataset	Best Architecture	Test RMSE	Test %RMSE
Dataset 1 (Univariate)	8 nodes	0.0851	3.424%
Dataset 2 (Bivariate)	4→4	0.0603	1.189%

5.2 Dataset 1: Univariate Regression

Figure 11 shows the training error of the selected 8-hidden-node architecture for the univariate regression task (Dataset 1). The x-axis represents the training epochs, while the y-axis represents the average training error returned after each epoch.

The plot is generated by recording the average training error returned by `trainEpoch()` at the end of each epoch. The resulting sequence of errors is stored in the training history and plotted against the corresponding epoch numbers. Validation error is tracked separately during training for early stopping and is not shown in this figure.

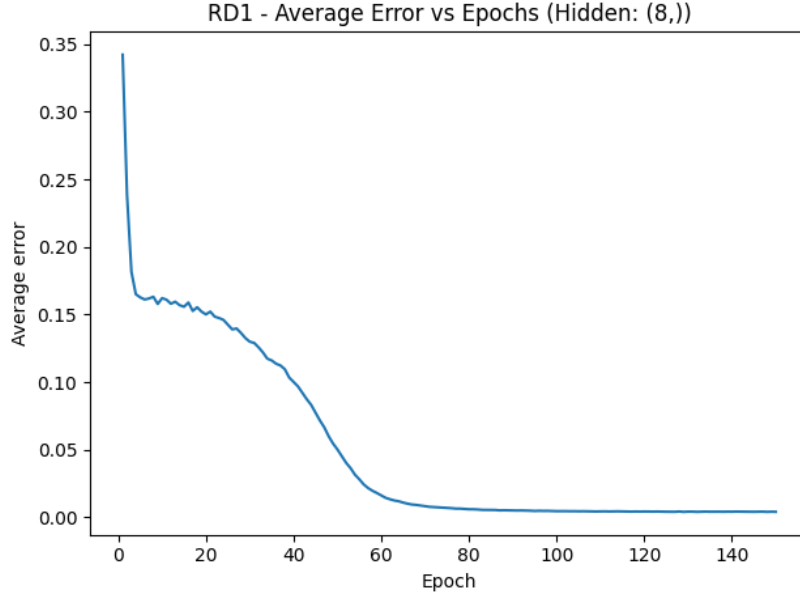


Figure 11: Dataset 1: average training error vs. epochs, best architecture (8 hidden nodes).

Figure 12 shows the model output superimposed on the target output for the training, validation, and test datasets using the best architecture selected through validation, which for Dataset 1 is the 8-hidden-node architecture.

Since Dataset 1 is a univariate regression task, the x-axis represents the input feature x , while the y-axis represents the target value y and the corresponding model prediction. For each split, the actual target values and model predictions are plotted together against the input x -values.

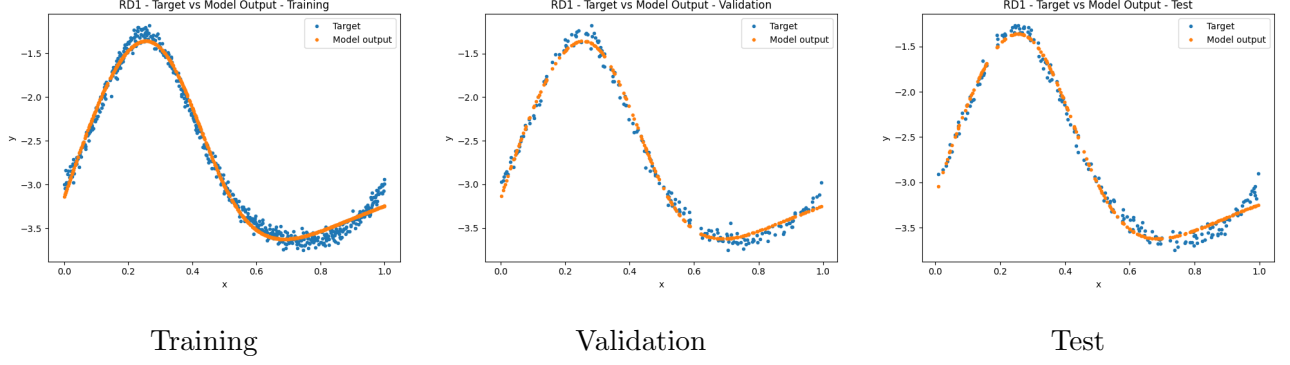


Figure 12: Dataset 1: model output superimposed on target y (8 hidden nodes).

Figure 13 shows scatter plots of the actual target values against the model predictions for the training, validation, and test datasets using the best-performing 8-hidden-node architecture. The x-axis represents the actual target value, while the y-axis represents the corresponding model output. The dashed diagonal line represents the ideal relationship $y = x$, where the predicted value is equal to the actual target value.

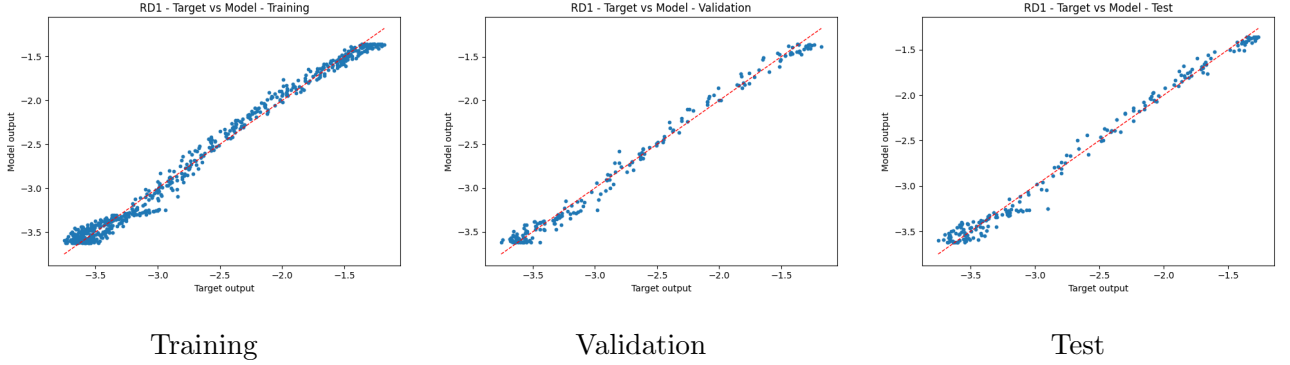


Figure 13: Dataset 1: scatter plot of target vs. model output (8 hidden nodes).

Figure 14 visualizes the activations of a representative hidden node and the output node for the selected 8-hidden-node architecture on the training, validation, and test datasets. The top row shows the activation of Hidden Layer 1, Node 0, while the bottom row shows the activation of the output node across the three dataset splits.

For Dataset 1, since the network has a single input feature, the horizontal axis represents the input feature x , while the vertical axis represents the corresponding node output (activation). The plots are generated by evaluating the selected network on the input values of each dataset split and recording the activation produced by the respective node.

The remaining node-activation plots for the other hidden nodes are provided in Appendix A for completeness.

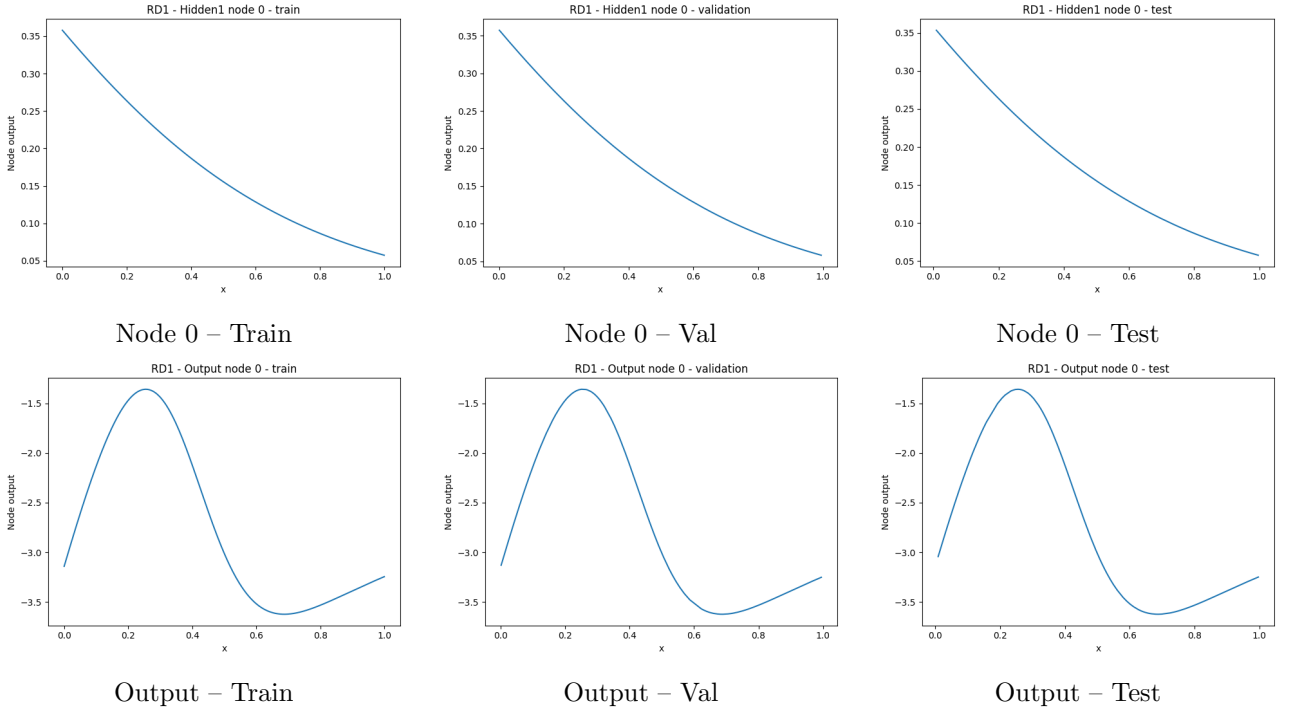


Figure 14: Dataset 1: representative hidden node (top) and output node (bottom) surfaces. Remaining nodes in Appendix A.

5.3 Dataset 2: Bivariate Regression

Figure 15 illustrates the training convergence of the optimal two-hidden-layer ($4 \rightarrow 4$) architecture for the bivariate regression task (Dataset 2).

The x-axis represents the training epochs, while the y-axis represents the average squared error computed during each epoch. The plot demonstrates rapid convergence, with the error decreasing sharply during the initial epochs and reaching a near-zero plateau within the first ten epochs. The error then remains highly stable for the remainder of the training process.

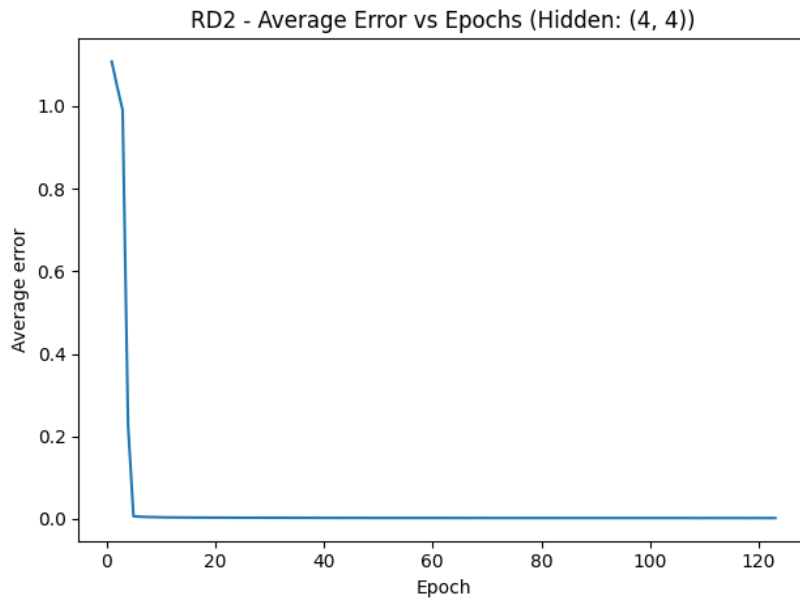


Figure 15: Dataset 2: average training error vs. epochs, best architecture ($4 \rightarrow 4$).

Figure 16 shows the model predictions overlaid with the actual target values for the training, validation, and test datasets using the optimal two-hidden-layer ($4 \rightarrow 4$) architecture selected during validation.

Since Dataset 2 has two input features, the plots are three-dimensional, with x_1 and x_2 representing the input features and y representing the target variable. Both the ground-truth values and the corresponding model predictions are shown for each dataset split.

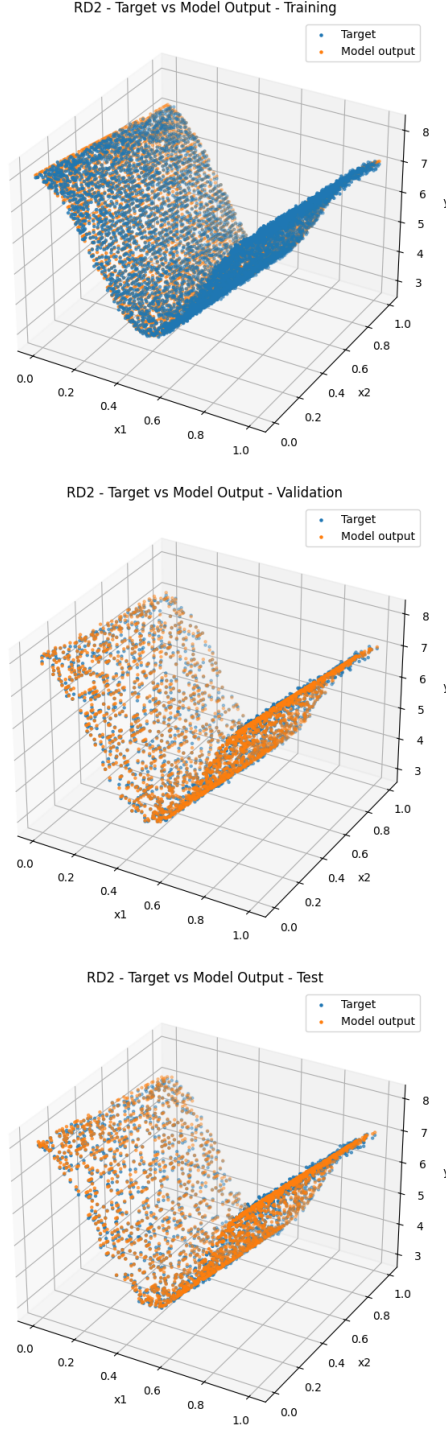


Figure 16: Dataset 2: model output superimposed on target y along x_1 and x_2 ($4 \rightarrow 4$).

Figure 17 presents scatter plots of the target values against the corresponding model outputs for the training, validation, and test datasets using the optimal ($4 \rightarrow 4$) architecture. The horizontal axis

represents the target value, while the vertical axis represents the model output. The three plots show the target–output pairs for the respective dataset splits.

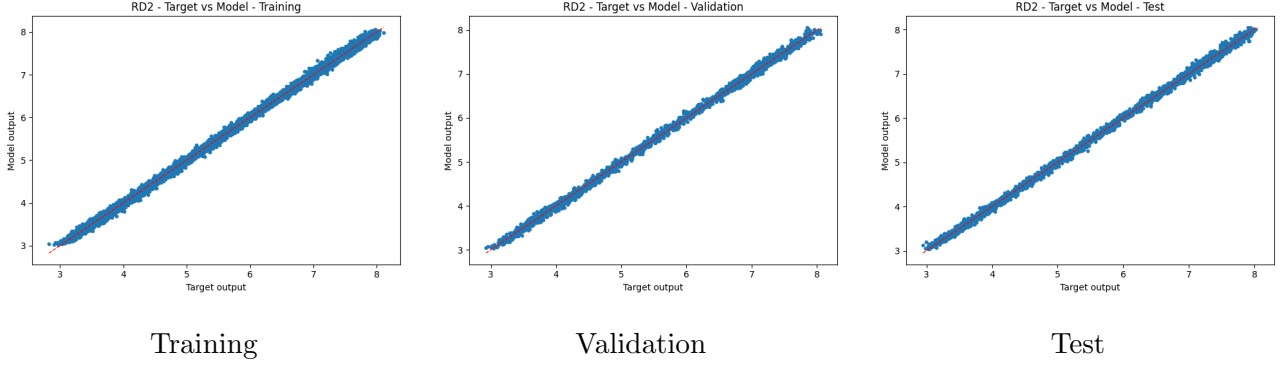


Figure 17: Dataset 2: scatter plot of target vs. model output ($4 \rightarrow 4$).

Figure 18 presents the node-output surfaces for representative nodes from the optimal ($4 \rightarrow 4$) architecture across the training, validation, and test datasets. The first row corresponds to Node 0 of Hidden Layer 1, the second row to Node 0 of Hidden Layer 2, and the third row to the output node.

Since Dataset 2 has two input features, the horizontal axes represent x_1 and x_2 , while the vertical axis represents the output of the corresponding node. Each surface therefore shows how the activation of that particular node varies with the two input features for the respective dataset split. The Hidden Layer 1 and Hidden Layer 2 plots show the intermediate representations produced by the corresponding hidden nodes, while the output-node plots show the final network output as a function of the two input features.

The remaining 21 node-output plots, covering the other nodes of the selected architecture across the training, validation, and test datasets, are provided in Appendix C.

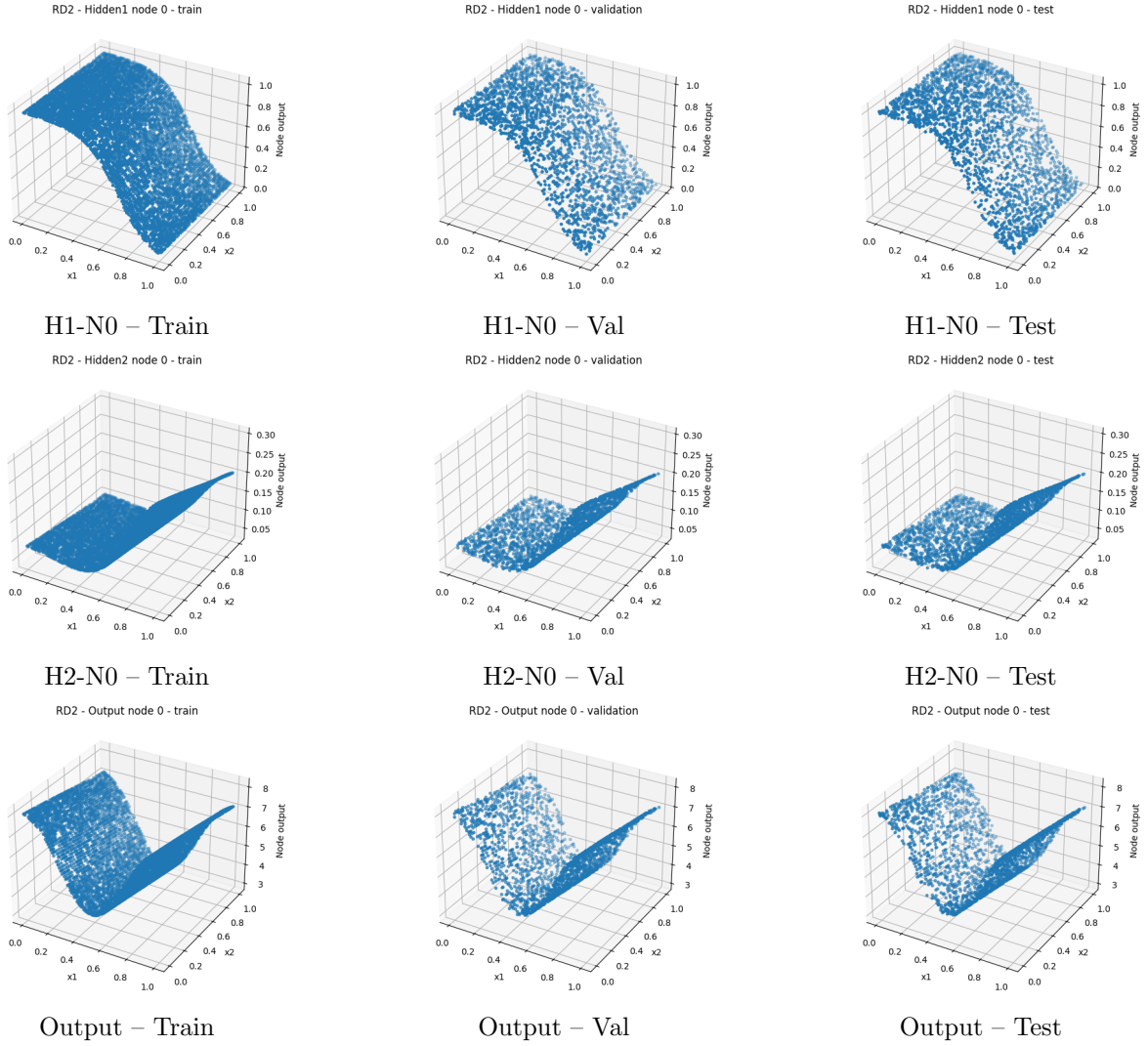


Figure 18: Dataset 2: representative H1, H2, and output node surfaces (4→4). Remaining nodes in Appendix C.

5.4 Comparison with Assignment 1 — Regression

Table 18: Regression performance: Assignment 1 (single neuron) vs. Assignment 2 (FCNN).

Dataset	Model	Test RMSE	Test %RMSE
Dataset 1 (Univariate)	Single Neuron (A1)	0.5291	19.29%
	FCNN 8-node (A2)	0.0851	3.424%
Dataset 2 (Bivariate)	Single Neuron (A1)	1.4251	25.85%
	FCNN 4→4 (A2)	0.0603	1.189%

The single-neuron model fits only a straight line (univariate) or a flat plane (bivariate). Both targets are strongly nonlinear and the linear model’s error is dominated by systematic bias. The FCNN, by composing sigmoid hidden activations, approximates the underlying curves and surfaces, reducing test %RMSE to $\approx 1.1\%$ — a level fundamentally unachievable by any linear model.

5.5 Inference and Observations from Regression Results

- Both error curves show a steep decrease during the initial epochs followed by a stable plateau, indicating that the training error decreases rapidly and subsequently stabilises.
- The target-versus-model-output scatter plots for the training, validation, and test datasets show similar distributions, with the predictions remaining close to the corresponding target values across the three splits.
- The output-overlay plots show that the selected FCNN architectures reproduce the underlying target functions for both regression datasets. For Dataset 1, the predicted outputs follow the variation of the target function, while for Dataset 2, the predicted surface follows the variation of the target over the two-dimensional input space.
- The validation results show that the performance varies with the number of hidden nodes and layers. For Dataset 1, the selected architecture is determined by comparing the candidate hidden-layer sizes, while for Dataset 2, the two-hidden-layer ($4 \rightarrow 4$) architecture provides the best validation performance among the evaluated architectures.
- The node-output plots illustrate the intermediate activations produced by the hidden layers and the final output of the network. The hidden-node surfaces vary smoothly over the input domain, while the output-node surface represents the final mapping from the input features to the predicted target.

6 Conclusion

This assignment demonstrated the effect of hidden-layer architecture on both classification and regression tasks. For the linearly separable dataset, the 2-node FCNN achieved perfect classification accuracy, matching the performance of the multi-classifier system used in Assignment 1 while providing a single unified network.

For the nonlinearly separable dataset, the smaller architectures, up to $(16, 8)$, converged to degenerate solutions that predominantly predicted a single class. In contrast, the $(32, 16)$ architecture achieved 99.67% test accuracy, substantially improving upon the 47.17% accuracy obtained by the best single-neuron baseline. This result highlights the importance of sufficient hidden-layer capacity when learning more complex nonlinear decision boundaries under the given loss and activation functions.

For both regression datasets, the selected FCNN architectures achieved test %RMSE of 3.424% (univariate) and 1.189% (bivariate), demonstrating their ability to approximate nonlinear target functions using sigmoid hidden layers.

Validation-based architecture selection was important throughout the experiments. The regression results showed that intermediate and larger architectures did not necessarily provide better performance, while for the nonlinear classification task, the choice of architecture had a substantial effect on the final classification performance. Overall, the experiments demonstrate that hidden-layer architecture and network capacity play an important role in determining the performance of FCNNs across different types of learning problems.

A Dataset 1 Regression: Remaining Hidden Node Surfaces

This appendix provides the remaining 21 node-activation plots corresponding to the selected 8-hidden-node architecture across the training, validation, and test datasets. Together with the six plots presented in Figure 14, these constitute the complete set of 27 node-activation plots required for the selected architecture.

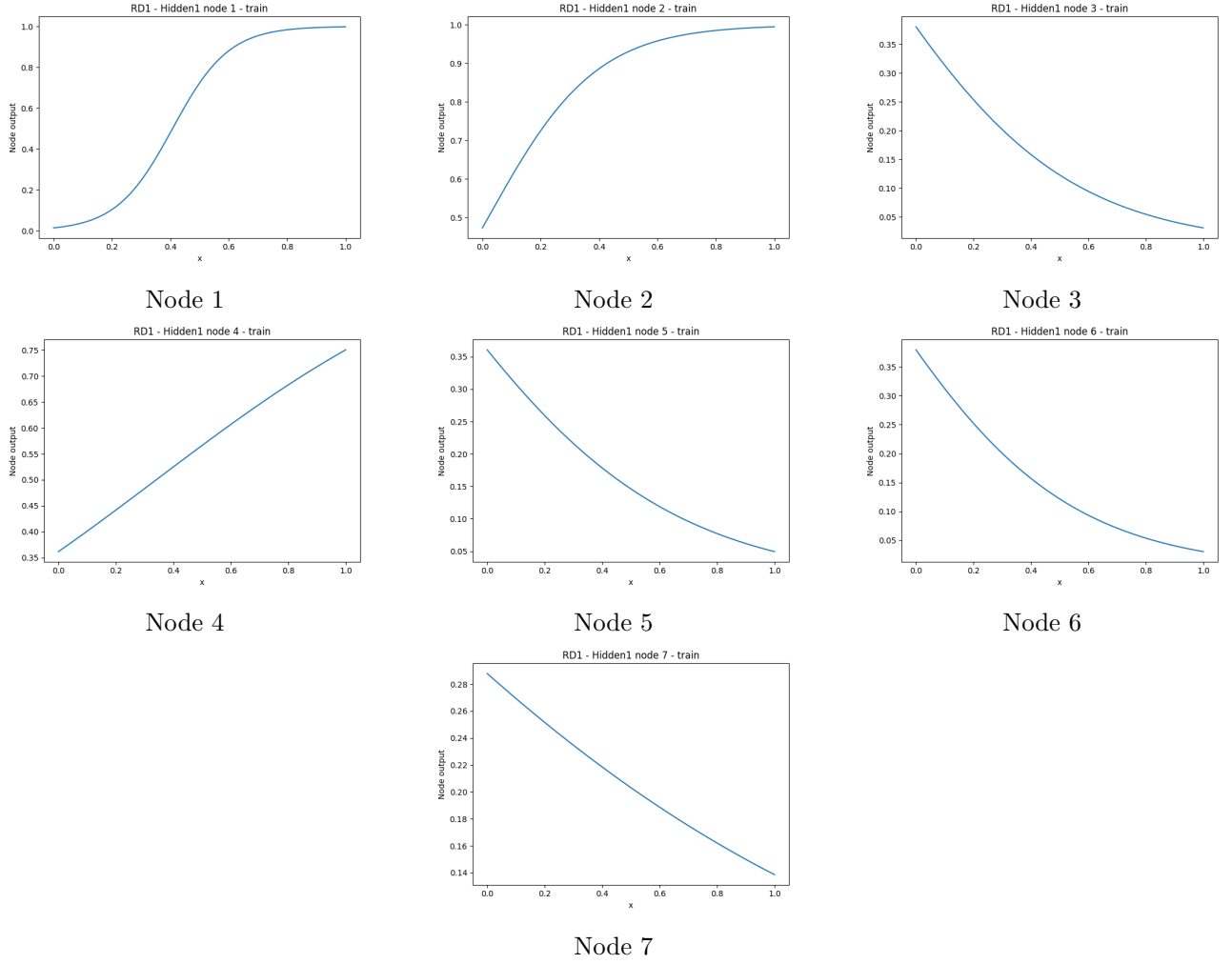


Figure 19: Dataset 1: hidden nodes 1–7 on training data (8-node architecture).

B Appendix: NLS Dataset 2 Additional Node Surfaces (32,16)

The (32,16) architecture produces 32 first-layer nodes, 16 second-layer nodes, and 3 output nodes per data split. A representative selection of the first few nodes is shown in the main body (Figures 8–10). To provide a comprehensive yet readable overview of the internal network representations, this appendix displays a sequential subsample of the remaining hidden nodes (sampled in steps of 5) across all three data splits (training, validation, and testing).

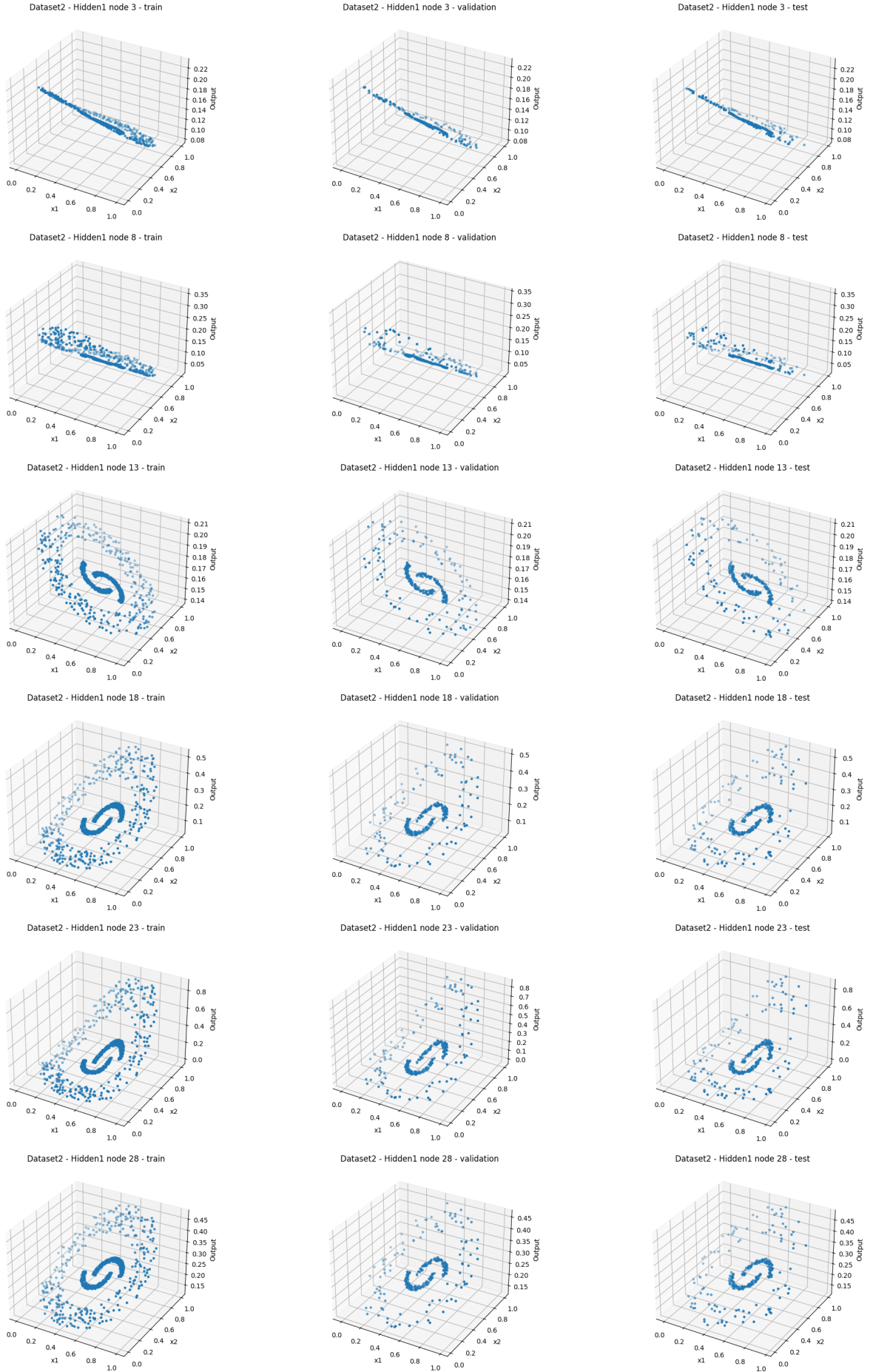


Figure 20: NLS (32,16): Hidden Layer 1 sequential node surfaces for train (left), validation (centre), and test (right) splits.

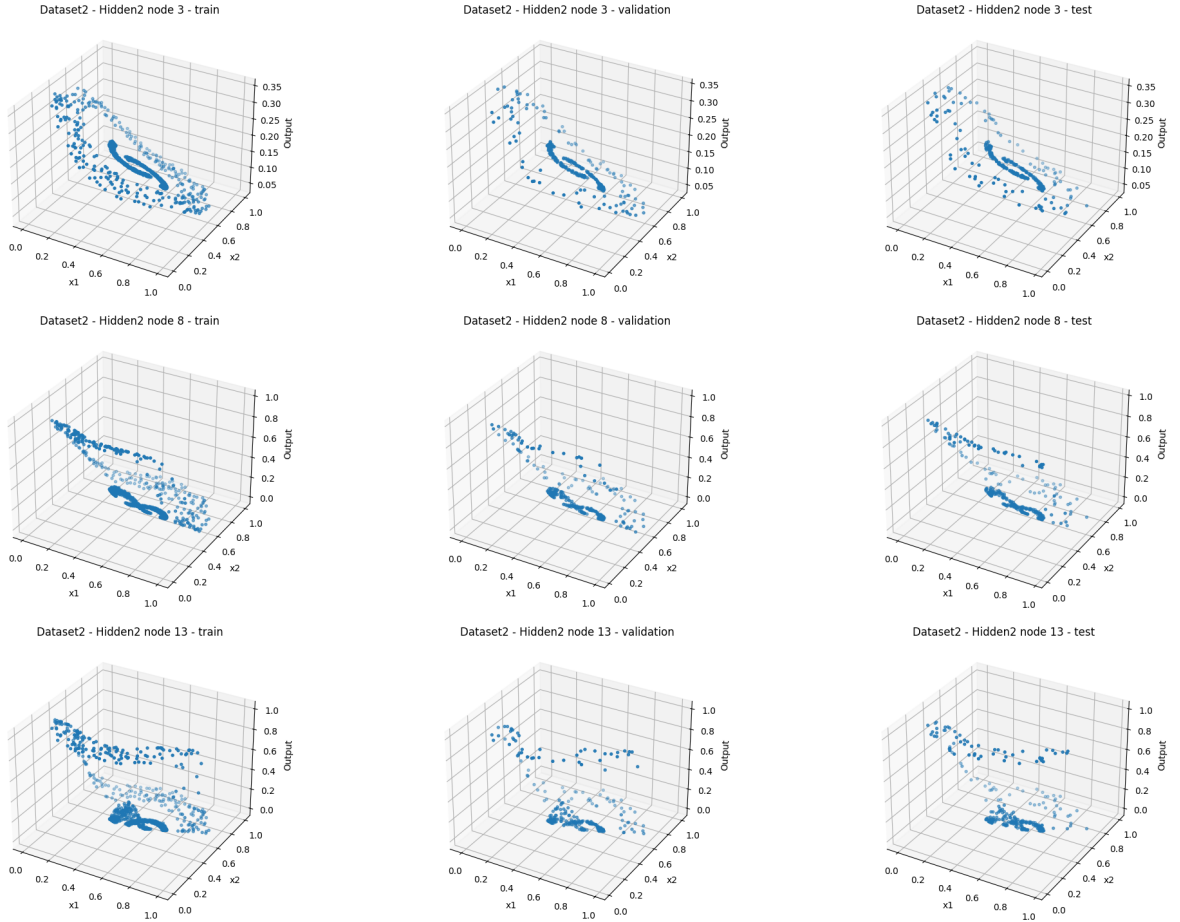


Figure 21: NLS (32,16): Hidden Layer 2 sequential node surfaces for train (left), validation (centre), and test (right) splits.

C Dataset 2 Regression: Remaining Hidden Node Surfaces

The two-hidden-layer ($4 \rightarrow 4$) architecture produces 4 nodes in the first hidden layer, 4 nodes in the second hidden layer, and 1 output node per data split. Assuming the representative first nodes (e.g., H1-Node 0 and Output) are shown in the main text, the remaining 21 node activation surfaces across the training, validation, and test splits are presented below.

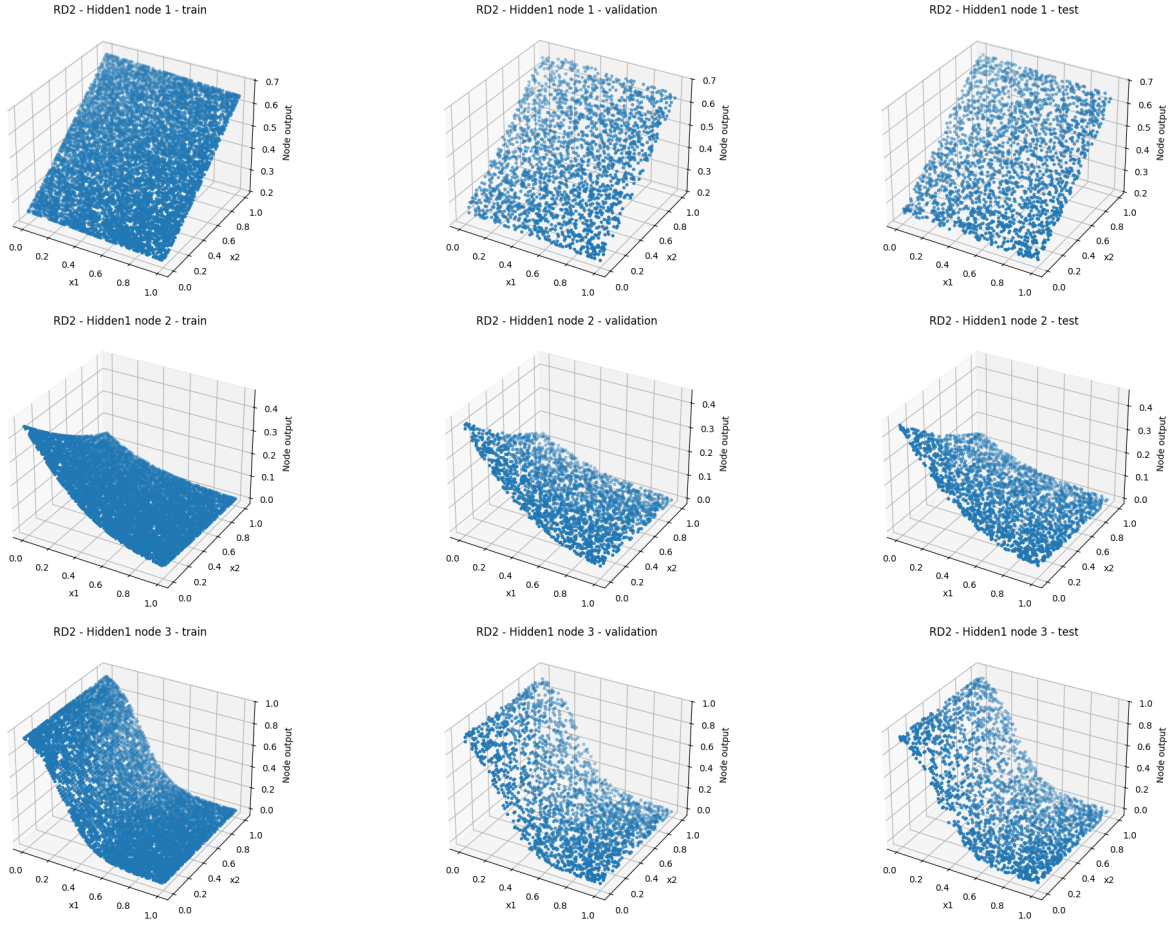


Figure 22: Dataset 2 (4→4): Remaining Hidden Layer 1 node surfaces (Nodes 1–3). Columns from left to right: Training, Validation, and Test data.

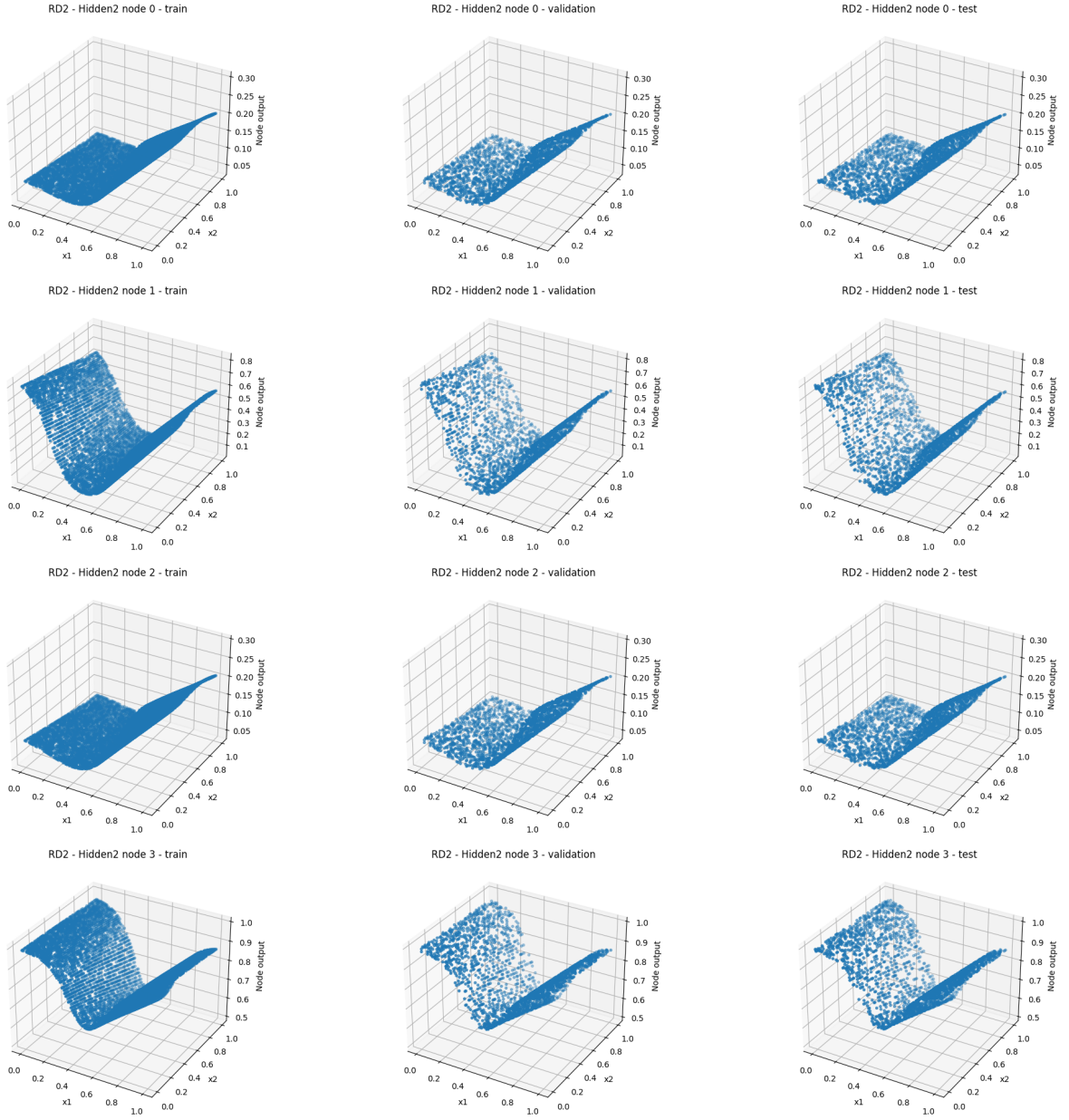


Figure 23: Dataset 2 (4→4): Complete Hidden Layer 2 node surfaces (Nodes 0–3). Columns from left to right: Training, Validation, and Test data.